

1. Write a menu driven program to implement queue operations such as enqueue, dequeue, peek, Display elements, isEmpty, isFull using static array.

Source Code:

```
#include <stdio.h>

int size = 5;
int queue[5];
int front = -1;
int rear = -1;

void enqueue(int value){
    if (rear == size-1){
        printf("OVERFLOW\n");
        return;
    } else if (front == -1 && rear == -1){
        front++;
        rear++;
    } else {
        rear++;
    }
    queue[rear] = value;
}

int dequeue(){
    int temp;
    if (front == -1 && rear == -1){
        printf("UNDERFLOW\n");
        return -1;
    } else if (front == rear){
        temp = queue[front];
        front = -1;
        rear = -1;
        return temp;
    } else {
        temp = queue[front];
        front++;
        return temp;
    }
}

void peek(){
    if (front == -1 && rear == -1){
        printf("Queue is Empty\n");
        return;
    } else {
        printf("First Element: %d\n", queue[front]);
    }
}

void display(){
    if (front == -1 && rear == -1){
        printf("Queue is Empty\n");
    }
```

```

        return;
    } else {
        for (int i = front; i<=rear; i++){
            printf("%d ", queue[i]);
        }
    }
    printf("\n");
}

int isEmpty(){
    if (front == -1 && rear == -1) return 1;
    else return 0;
}

int isFull(){
    if (rear == size-1) return 1;
    else return 0;
}

int main(){
    int choice, value;
    printf("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. IsEmpty\n6. IsFull\n7. Exit\n");
    while(1){
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                printf("Dequeued Element: %d\n", dequeue());
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                if(isEmpty()) printf("Queue is Empty\n");
                else printf("Queue is not Empty\n");
                break;
            case 6:
                if(isFull()) printf("Queue is Full\n");
                else printf("Queue is not Full\n");
                break;
            case 7:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}

```

```
    return 0;  
}
```

Output:

```
PS D:\DS\DSL\LAB6> gcc p1.c -o p1.exe  
PS D:\DS\DSL\LAB6> .\p1.exe  
1. Enqueue  
2. Dequeue  
3. Peek  
4. Display  
5. IsEmpty  
6. IsFull  
7. Exit  
Enter your choice: 5  
Queue is Empty  
Enter your choice: 1  
Enter value to enqueue: 10  
Enter your choice: 1  
Enter value to enqueue: 20  
Enter your choice: 1  
Enter value to enqueue: 30  
Enter your choice: 4  
10 20 30  
Enter your choice: 3  
First Element: 10  
Enter your choice: 2  
Dequeued Element: 10  
Enter your choice: 4  
20 30  
Enter your choice: 1  
Enter value to enqueue: 40  
Enter your choice: 1  
Enter value to enqueue: 50  
Enter your choice: 1  
Enter value to enqueue: 60  
Enter your choice: 6  
Queue is Full  
Enter your choice: 1  
Enter value to enqueue: 70  
OVERFLOW  
Enter your choice: 7  
PS D:\DS\DSL\LAB6>
```

2. Write a menu driven program to implement queue operations such as enqueue, dequeue, peek, Display elements, isEmpty using linked list.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *front = NULL;
struct node *rear = NULL;

void enqueue(int value){
    struct node *new;
    new = (struct node *) malloc(sizeof(struct node));
    new->data = value;
    new->next = NULL;

    if (front == NULL && rear == NULL){
        front = new;
        rear = new;
    } else {
        rear->next = new;
        rear = new;
    }
}

void dequeue(){
    struct node *temp;
    if (front == NULL && rear == NULL){
        printf("UNDERFLOW\n");
        return;
    } else if (front == rear){
        temp = front;
        printf("Dequeued Element: %d\n", temp->data);
        front = NULL;
        rear = NULL;
        free(temp);
    } else {
        temp = front;
        printf("Dequeued Element: %d\n", temp->data);
        front = front->next;
        free(temp);
    }
}

void peek(){
    if (front == NULL && rear == NULL){
        printf("Queue is Empty\n");
        return;
    }
}
```

```

    } else {
        printf("First Element: %d\n", front->data);
    }
}

void display(){
    if (front == NULL && rear == NULL){
        printf("Queue is Empty\n");
        return;
    } else {
        struct node *ptr = front;
        while( ptr != NULL){
            printf("%d ", ptr->data);
            ptr = ptr->next;
        }
    }
    printf("\n");
}

int isEmpty(){
    if (front == NULL && rear == NULL) return 1;
    else return 0;
}

int main(){
    int choice, value;
    printf("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. IsEmpty\n6. Exit\n");
    while(1){
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                if(isEmpty()) printf("Queue is Empty\n");
                else printf("Queue is not Empty\n");
                break;
            case 6:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}

```

```
}  
    return 0;  
}
```

Output:

```
PS D:\DS\DSL\LAB6> gcc p2.c -o p2.exe  
PS D:\DS\DSL\LAB6> .\p2.exe  
1. Enqueue  
2. Dequeue  
3. Peek  
4. Display  
5. IsEmpty  
6. Exit  
Enter your choice: 5  
Queue is Empty  
Enter your choice: 1  
Enter value to enqueue: 10  
Enter your choice: 1  
Enter value to enqueue: 20  
Enter your choice: 1  
Enter value to enqueue: 30  
Enter your choice: 4  
10 20 30  
Enter your choice: 3  
First Element: 10  
Enter your choice: 2  
Dequeued Element: 10  
Enter your choice: 4  
20 30  
Enter your choice: 5  
Queue is not Empty  
Enter your choice: 6  
PS D:\DS\DSL\LAB6>
```

3. Write a menu driven program to implement circular queue operations such as enqueue, dequeue, peek, Display elements, isEmpty, isFull using static array.

Source Code:

```
#include <stdio.h>

int size = 5;
int circular_queue[5];
int front = -1;
int rear = -1;

void enqueue(int value){
    if ((rear+1)%size == front){
        printf("OVERFLOW\n");
        return;
    } else if (front == -1 && rear == -1){
        front++;
        rear++;
    } else {
        rear = (rear+1)%size;
    }
    circular_queue[rear] = value;
}

int dequeue(){
    int temp;
    if (front == -1 && rear == -1){
        printf("UNDERFLOW\n");
        return -1;
    } else if (front == rear){
        temp = circular_queue[front];
        front = -1;
        rear = -1;
        return temp;
    } else {
        temp = circular_queue[front];
        front = (front+1)%size;
        return temp;
    }
}

void peek(){
    if (front == -1 && rear == -1){
        printf("Queue is Empty\n");
        return;
    } else {
        printf("First Element: %d\n", circular_queue[front]);
    }
}

void display(){
    if (front == -1 && rear == -1){
        printf("Queue is Empty\n");
    }
```

```

        return;
    } else {
        for (int i = front; i!=rear; i = (i+1)%size){
            printf("%d ", circular_queue[i]);
        }
        printf("%d ", circular_queue[rear]);
    }
    printf("\n");
}

int isEmpty(){
    if (front == -1 && rear == -1) return 1;
    else return 0;
}

int isFull(){
    if ((rear+1)%size == front) return 1;
    else return 0;
}

int main(){
    int choice, value;
    printf("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. IsEmpty\n6. IsFull\n7. Exit\n");
    while(1){
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;
            case 2:
                printf("Dequeued Element: %d\n", dequeue());
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                if(isEmpty()) printf("Queue is Empty\n");
                else printf("Queue is not Empty\n");
                break;
            case 6:
                if(isFull()) printf("Queue is Full\n");
                else printf("Queue is not Full\n");
                break;
            case 7:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
}

```

```
}  
    return 0;  
}
```

Output:

```
PS D:\DS\DSL\LAB6> gcc p3.c -o p3.exe  
PS D:\DS\DSL\LAB6> .\p3.exe  
1. Enqueue  
2. Dequeue  
3. Peek  
4. Display  
5. IsEmpty  
6. IsFull  
7. Exit  
Enter your choice: 5  
Queue is Empty  
Enter your choice: 1  
Enter value to enqueue: 10  
Enter your choice: 1  
Enter value to enqueue: 20  
Enter your choice: 1  
Enter value to enqueue: 30  
Enter your choice: 4  
10 20 30  
Enter your choice: 2  
Dequeued Element: 10  
Enter your choice: 1  
Enter value to enqueue: 40  
Enter your choice: 1  
Enter value to enqueue: 50  
Enter your choice: 1  
Enter value to enqueue: 60  
Enter your choice: 6  
Queue is Full  
Enter your choice: 4  
20 30 40 50 60  
Enter your choice: 1  
Enter value to enqueue: 70  
OVERFLOW  
Enter your choice: 7  
PS D:\DS\DSL\LAB6>
```

4. Write a menu driven program to implement priority queue operations such as enqueue, dequeue, peek, Display elements, isEmpty using linked list.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int priority;
    int data;
    struct node *next;
};

struct node *front = NULL;

void enqueue(int p, int value){
    struct node *new = (struct node *) malloc(sizeof(struct node));
    new->priority = p;
    new->data = value;
    new->next = NULL;

    if (front == NULL || p < front->priority){
        new->next = front;
        front = new;
    } else {
        struct node *ptr = front;
        while (ptr->next != NULL && ptr->next->priority <= p){
            ptr = ptr->next;
        }
        new->next = ptr->next;
        ptr->next = new;
    }
}

void dequeue(){
    if (front == NULL){
        printf("UNDERFLOW\n");
        return;
    }
    struct node *temp = front;
    printf("Dequeued Element: %d\n", temp->data);
    front = front->next;
    free(temp);
}

void peek(){
    if (front == NULL){
        printf("Queue is Empty\n");
        return;
    }
    printf("First Element: %d\n", front->data);
}
```

```

void display(){
    if (front == NULL){
        printf("Queue is Empty\n");
        return;
    }
    struct node *ptr = front;
    while (ptr != NULL){
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

int isEmpty(){
    if (front == NULL) return 1;
    else return 0;
}

int main(){
    int choice;
    int value, priority;
    printf("1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. IsEmpty\n6. Exit\n");
    while(1){
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice){
            case 1:
                printf("Enter value's priority: ");
                scanf("%d", &priority);
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(priority, value);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                if(isEmpty()) printf("Queue is Empty\n");
                else printf("Queue is not Empty\n");
                break;
            case 6:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}

```

Output:

```
PS D:\DS\DSL\LAB6> gcc p4.c -o p4.exe
PS D:\DS\DSL\LAB6> .\p4.exe
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. IsEmpty
6. Exit
Enter your choice: 5
Queue is Empty
Enter your choice: 1
Enter value's priority: 2
Enter value to enqueue: 20
Enter your choice: 1
Enter value's priority: 1
Enter value to enqueue: 10
Enter your choice: 1
Enter value's priority: 3
Enter value to enqueue: 30
Enter your choice: 4
10 20 30
Enter your choice: 3
First Element: 10
Enter your choice: 2
Dequeued Element: 10
Enter your choice: 4
20 30
Enter your choice: 2
Dequeued Element: 20
Enter your choice: 2
Dequeued Element: 30
Enter your choice: 2
UNDERFLOW
Enter your choice: 5
Queue is Empty
Enter your choice: 6
PS D:\DS\DSL\LAB6>
```